

Predicting Clinical Trial Completion from ISRCTN Registry Data

Executive Summary

Project snapshot	Healthcare machine learning case study
Problem	Predict whether a registered clinical trial will complete or terminate early.
Data	ISRCTN registry data; 27,506 records and 29 working columns after initial pruning. Source Link: ISRCTN Advanced Search Results
Target	Completed = 1, Not Completed = 0, with an approximate 96:4 class split.
Stack	Snowflake preprocessing, Python feature engineering, SciBERT embeddings, MLflow tracking.
Outcome	XGBoost delivered the best overall macro F1 baseline, with further scope for minority-class tuning.

This project asked a practical healthcare machine learning question: can information available at the point of trial registration be used to estimate whether a clinical study will complete or terminate early? I built an end-to-end workflow around ISRCTN (a primary clinical study registry recognised by the World Health Organisation) registry data, combining upstream SQL preprocessing in Snowflake, local feature engineering in Python, semantic text encoding with SciBERT, model benchmarking across several classical algorithms, and experiment tracking in MLflow. The final output was not just a trained model, but a reproducible workflow that could be audited, tuned and packaged for deployment.

The technical challenge was less about obtaining a high headline accuracy score and more about making sensible modelling decisions in a noisy, imbalanced dataset. The working dataset of 27,506 records and 29 columns after initial pruning, with 10.27% of cells empty overall. Missingness was especially high in fields such as funding type, drug or device names, phase and age limits, so the project required careful decisions about which fields to keep, which to engineer, and which to leave blank without forcing unreliable imputations.

The Business Context and Objective

This project addresses a practical problem in UK commercial clinical research: identifying, as early as possible, which trials are at risk of failing to complete. That matters because the wider system has been under sustained pressure, with slow and complex trial set-up, stretched NHS research capacity, and declining patient

recruitment making the UK a harder environment in which to deliver commercial studies successfully. Even where trial starts have begun to recover, participation has continued to fall, which means the challenge is no longer just attracting studies but delivering them efficiently and seeing them through to completion. Against that backdrop, this project uses ISRCTN registration data to predict trial outcome from information available near the start of the process, reframing the task as Completed vs Not Completed by excluding ongoing trials and grouping stopped, deferred and suspended studies together. In effect, the model attempts to turn a broader delivery problem into an early-warning tool, helping identify the minority of studies most likely to fail so that sponsors or research teams could intervene sooner and allocate support more effectively.

Project Context and Data Design

A strong feature of this project was the decision to push simple but important transformations upstream. I removed unnecessary columns in Snowflake before any heavier local processing began, which was both computationally cheaper and cleaner from a pipeline design perspective. That decision also made sense from a modelling standpoint. Many fields in a clinical trial registry are administrative, high-cardinality, or only populated once a study has materially progressed. While such variables may be useful for record keeping, they are not always appropriate for a predictive model that is meant to operate early in the trial lifecycle.

With that in mind, I excluded administrative identifiers, registry cross-references, document attachments, and post-trial artefacts such as results files, publications, and protocol uploads. More importantly, I removed variables such as `REASON_ABANDONED` and `TOTAL_FINAL_ENROLMENT` because they leak the target. If a model can see evidence that only appears after a study has already failed or concluded, then it is no longer learning a meaningful early-warning signal; it is simply reading the future.

Target Definition and Class Imbalance

The target definition was another important design choice. Rather than modelling every study status as a separate category, I reframed the problem as a binary classification task: Completed versus Not Completed. I excluded ongoing studies because they had not yet reached a final outcome, and I grouped stopped, deferred, and suspended studies into a single Not Completed class. This produced a class balance of roughly 96:4.

That imbalance had important implications for how I evaluated model performance. In a dataset like this, accuracy can be a misleading metric, because a model can appear to perform well simply by predicting the majority class most of the time. For that reason, I treated macro F1 as the main comparison metric and also examined precision and recall separately for each class. Completed trials were encoded as class 1, while Not Completed trials were encoded as class 0.

Because the underlying business interest was failure detection, class 0 was the operationally important class when interpreting recall and ROC AUC. In other words, the real test was not whether the model could correctly identify the many studies that completed, but whether it could reliably surface the smaller set of trials that did not. That distinction mattered, because it kept the evaluation aligned with the actual decision-making value of the model rather than with a superficially strong aggregate score.

Feature Engineering and Representation

The project combined structured and unstructured information in a sensible way. Date fields were converted into durations relative to a reference date so that the model learned time intervals rather than raw calendar values. Related text fields were grouped into broader themes to keep the feature space manageable and to preserve context. In practice, that meant representing both what the study is about and how it is designed, rather than relying solely on narrow one-hot encoded fragments.

For language features, I used the base SciBERT model rather than a generic encoder. That was a good fit for scientific and clinical vocabulary, and it kept the solution lightweight enough to run locally. The text pipeline converted strings to tokens, generated embeddings, and then flattened the 768-dimensional output into model-ready columns so the encoded features could be merged with the wider tabular dataset. I then normalised the encoded values before modelling.

Model Development Approach

I benchmarked a range of algorithms rather than assuming one model family would win. The comparison included tree-based ensembles, a linear baseline, a support vector classifier, K-nearest neighbours and XGBoost. All experiments were logged in MLflow so that parameters, metrics and artefacts were recorded consistently.

Selected Metrics from the MLflow Dashboard

The figures below in Table 1 are supported by the selected metrics from the MLflow dashboard. This table highlights the most useful comparison for the business problem: not only overall score, but also how each model behaves on the minority Not Completed class.

Tab 1. MLflow Dashboard: Initial Accuracy and Marco F1 scores

Model	Accuracy	Macro F1
XGBoost	0.96	0.59
Extra Trees	0.89	0.57
Random Forest	0.80	0.57
Gradient Boosting	0.86	0.56
Logistic Regression	0.87	0.55
Support Vector Classifier	0.82	0.54
K-Nearest Neighbours	0.90	0.58
Decision Tree	0.76	0.48

Interpretation of the Results

The first MLflow dashboard shows that XGBoost achieved the strongest overall macro F1 score at 0.59, narrowly ahead of KNN at 0.58 and Random Forest and Extra Trees at 0.57. Accuracy tells a much less helpful story, with XGBoost reaching 0.96 mainly because the dataset is dominated by completed trials. See Figure 1 for more information.

Figure 1. MLflow Dashboard: Macro F1 comparison



XGBoost leads with 0.59, but only by a narrow margin over KNN and the ensemble tree methods. That closeness is useful in a portfolio context because it shows the final choice was evidence-led rather than obvious from the outset.

Hyperparameter Tuning and Deployment

As documented within MLflow, the parent study successfully completed 30 distinct trials over a duration of approximately 1.6 hours, progressively converging upon an XGBoost configuration that distinctly favoured a controlled, meticulously regularised learning paradigm, as opposed to an overly aggressive fit. Furthermore, the optimal setup incorporated 850 estimators with a restricted maximum tree depth of 5 and a markedly low learning rate of 0.0055, whilst both subsampling and column sampling parameters were deliberately maintained marginally below 0.9 in order to significantly improve algorithmic generalisation.

Additional regularisation was effectively achieved through the application of a `min_child_weight` of 14.85, a `reg_alpha` of 2.03, and a `reg_lambda` of 2.74. Thus, this suggests that the search algorithm was actively rewarding systemic robustness within the context of a inherently noisy, highly imbalanced classification task, rather than simply prioritising raw model complexity. Subsequently, the most successful child run categorically confirmed this strategic choice, meticulously logging an exact log loss of 0.131 whilst utilising a standard `binary:logistic` objective function, with log loss explicitly designated as the primary evaluation metric.

Therefore, the subsequent phase of deployment followed a natural, logical progression. Once I had definitively isolated the winning run within MLflow, I ensured that the finalised model was packaged using Docker. This methodical approach guaranteed that the operational architecture could be reliably reproduced and consistently served well outside the initial confines of the analytical notebook environment.

Docker Container Link: <https://hub.docker.com/r/diiocker/my-mlflow-model>

Limitations and Next Steps

Should this project be extended further, the primary priority would be to optimise more directly for the operational outcome. Given that the fundamental aim is to flag clinical trials that are statistically unlikely to reach completion, I would systematically tune the decision threshold against class-0 recall, the precision-recall balance, or an explicitly defined cost function, rather than relying upon the default threshold inherently derived from a standard log-loss objective. Furthermore, it would be prudent to rigorously test class weighting, focal-style loss functions, and calibrated probability outputs in order to ascertain whether minority-class discrimination could be actively improved without consequently collapsing the model's precision.